VIjeth Madhwaraj Kadambalithaya I section
PES1UG24CS531

SCREEN SHOTS:

PHASE 1:

```
ijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ make test_objects
cc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
cc -Wall -Wextra -O2 -c object.c -o object.o
bject.c: In function ‘object_write’:
bject.c:147:47: warning: ‘/tmpXXXXXX’ directive output may be truncated writing 10 bytes into a region of size between 1 and 512 [-Wformat-truncation=]
 147 |     snprintf(temp_path, sizeof(temp_path), "%s/tmpXXXXXX", dir_path);
     |                                             ^~~~~~~~~~
n file included from /usr/include/stdio.h:980,
                 from pes.h:8,
                 from object.c:11:
n function ‘snprintf’,
    inlined from ‘object_write’ at object.c:147:5:
usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: ‘__builtin___snprintf_chk’ output between 11 and 522 bytes into a destination of size 512
  54 |   return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
     |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  55 |                                    __glibc_objsize (__s), __fmt,
     |                                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  56 |                                    __va_arg_pack ());
     |                                    ~~~~~~~~~~~~~~~~~
cc -o test_objects test_objects.o object.o -lcrypto
ijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ ./test_objects
tored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
bject stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
ASS: blob storage
ASS: deduplication
ASS: integrity check

ll Phase 1 tests passed.
```

```
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function ‘object_write’:
object.c:147:47: warning: ‘/tmpXXXXXX’ directive output may be truncated writing 10 bytes into a region of size between 1 and 512 [-Wformat-truncation=]
  147 |     snprintf(temp_path, sizeof(temp_path), "%s/tmpXXXXXX", dir_path);
      |                                             ^~~~~~~~~~
In file included from /usr/include/stdio.h:980,
                 from pes.h:8,
                 from object.c:11:
In function ‘snprintf’,
    inlined from ‘object_write’ at object.c:147:5:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: ‘__builtin___snprintf_chk’ output between 11 and 522 bytes into a destination of size 512
   54 |   return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   55 |                                    __glibc_objsize (__s), __fmt,
      |                                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   56 |                                    __va_arg_pack ());
      |                                    ~~~~~~~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ .pes/objects -type f
bash: .pes/objects: Is a directory
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$
```

PHASE 2:

```
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ micro tree.c
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ make test_tree
Makefile:48: warning: overriding recipe for target 'test_tree'
Makefile:22: warning: ignoring old recipe for target 'test_tree'
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
tree.c: In function ‘write_tree_level’:
tree.c:94:15: warning: implicit declaration of function ‘object_write’ [-Wimplicit-function-declaration]
   94 |     int res = object_write(OBJ_TREE, data, len, out_id);
      |               ^~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization
```

```
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ find .pes/objects -type f
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ find .pes/objects -type f
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ ^C
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ find .pes/objects -type f
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ xxd .pes/objects/XX/YYY... | head -20
xxd: .pes/objects/XX/YYY...: No such file or directory
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ xxd .^C
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ xxd .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476 | head -20
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
```

PHASE 3:

```
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ micro index.c
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ make pes
Makefile:48: warning: overriding recipe for target 'test_tree'
Makefile:22: warning: ignoring old recipe for target 'test_tree'
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ echo "hello" > file1.txt
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ echo "world" > file2.txt
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ ./pes add file1.txt file2.txt
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.c
  untracked:  README.md
  untracked:  pes.h
  untracked:  test_sequence.sh
  untracked:  index.h
  untracked:  test_objects.c
  untracked:  test_tree
  untracked:  commit.c
  untracked:  Makefile
  untracked:  pes.c
  untracked:  object.c
  untracked:  test_tree.c
  untracked:  commit.h
  untracked:  tree.h
  untracked:  .gitignore
  untracked:  test_objects
  untracked:  index.c

vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ cat .pes/index    # Human-readable text format
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776681484 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776681487 6 file2.txt
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$
```

PHASE 4:

```
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ micro commit.c
vijeth@vijeth-VirtualBox:~/PES1UG24CS531-PES-vcs$ make test-integration
Makefile:48: warning: overriding recipe for target 'test_tree'
Makefile:22: warning: ignoring old recipe for target 'test_tree'
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 4d1428da1d8c... Initial commit

Log after first commit:
commit 4d1428da1d8c1eb673f702730fc61b134f934a3f995fc98ece120e778a0edcbf
Author: Vijeth Madhwaraj PES1UG24CS531
Date:   1776681813

    Initial commit
```

```
    Initial commit


--- Second Commit ---
Committed: b74508d9bae9... Update file.txt

--- Third Commit ---
Committed: dd14eef4dc2d... Add farewell

--- Full History ---
commit dd14eef4dc2de9b2424cddd46c30adbc6915732e1ac80f9b7983927a2da4ab59
Author: Vijeth Madhwaraj PES1UG24CS531
Date:   1776681813

    Add farewellver auth!

commit b74508d9bae9e6ef36118de4e140c0bb7388a0189164df8a88d8b71fc42e4af8
Author: Vijeth Madhwaraj PES1UG24CS531
Date:   1776681813

    Update file.txt auth!

commit 4d1428da1d8c1eb673f702730fc61b134f934a3f995fc98ece120e778a0edcbf
Author: Vijeth Madhwaraj PES1UG24CS531
Date:   1776681813

    Initial committial commitPESA


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
dd14eef4dc2de9b2424cddd46c30adbc6915732e1ac80f9b7983927a2da4ab59

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/4d/1428da1d8c1eb673f702730fc61b134f934a3f995fc98ece120e778a0edcbf
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/b7/4508d9bae9e6ef36118de4e140c0bb7388a0189164df8a88d8b71fc42e4af8
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
```

PHASE 5 & 6

Q5.1: A branch is a file in `.pes/refs/heads/` storing a commit hash, so `pes checkout <branch>` updates `.pes/HEAD` to point to that branch reference. Then the working directory is rebuilt from the target commit's tree by creating, deleting, and overwriting files to match that snapshot. The index must also align with this state. The operation is complex because it risks overwriting uncommitted user changes and must maintain consistency during file system updates.

Q5.2: Dirty state is detected by comparing working directory files with the index using metadata like size and modification time. Then each index entry is compared with the target branch's commit tree using object hashes. If a file differs from the index and also differs in the target branch, it is a conflict. In that case, checkout is refused to prevent overwriting uncommitted changes.

Q5.3: In detached HEAD, commits are created normally and HEAD moves to them, but no branch pointer is updated. Once you switch away, those commits become unreachable. They remain in the object store until garbage collected. Recovery is done by creating a new branch pointing to the latest commit hash.

Q6.1: Garbage collection uses mark-and-sweep: start from all branch heads and HEAD, traverse commits, trees, and blobs, and mark reachable objects using a hash set. Then delete all unmarked objects from storage. With 100k commits and 50 branches, only reachable history is visited, not all stored objects. Each reachable object is processed once.

Q6.2: GC running during commit can delete newly created objects before they are referenced, causing race conditions. A commit may try to reference an object that GC already removed. This leads to repository corruption. Git avoids this using locking, atomic updates, and grace periods so recently created or referenced objects are not collected immediately.